PHASE-1 PRACTICAL EXAM

Dockerized E-Commerce Application Deployment on AWS EC2

Technologies:

AWS EC2 | Docker | Docker Compose | React.js | Node.js | Express.js | PostgreSQL | Nginx

---

1. Project Objective

Objective

The objective of this project is to containerize and deploy a full-stack E-Commerce application on an AWS EC2 Ubuntu server using Docker and Docker Compose.

The application is divided into three tiers:

1. **Presentation Tier** – React.js application served using Nginx
2. **Application Tier** – Node.js and Express.js backend
3. **Data Tier** – PostgreSQL 16 database

Docker Compose is used to manage the application containers, networking, and persistent database storage.

Key objectives

- Deploy the application on AWS EC2.
- Containerize frontend, backend, and database services.
- Create communication between containers using Docker networking.
- Expose only the required application ports.
- Persist PostgreSQL data using a Docker volume.
- Configure application secrets using environment variables.
- Verify that all services are running correctly.
- Document the architecture and deployment process.

---

2. Project Overview

This project is a full-stack E-Commerce application originally structured as a monorepo containing a React frontend and Node.js/Express backend.

For the practical exam, the application was containerized and deployed on an AWS EC2 Ubuntu server.

Application features

- User registration
- User login
- GitHub OAuth authentication

- Product listing
- Product category filtering
- Shopping cart
- Order placement
- Checkout
- Stripe payment integration
- PostgreSQL data persistence

---

3. Technology Stack

Component	Technology
Cloud	AWS EC2
OS	Ubuntu
Frontend	React.js
Frontend Web Server	Nginx
Backend	Node.js
API Framework	Express.js
Database	PostgreSQL 16
Containerization	Docker
Orchestration	Docker Compose
Authentication	Passport.js + GitHub OAuth
Payment	Stripe
Source Control	Git/GitHub

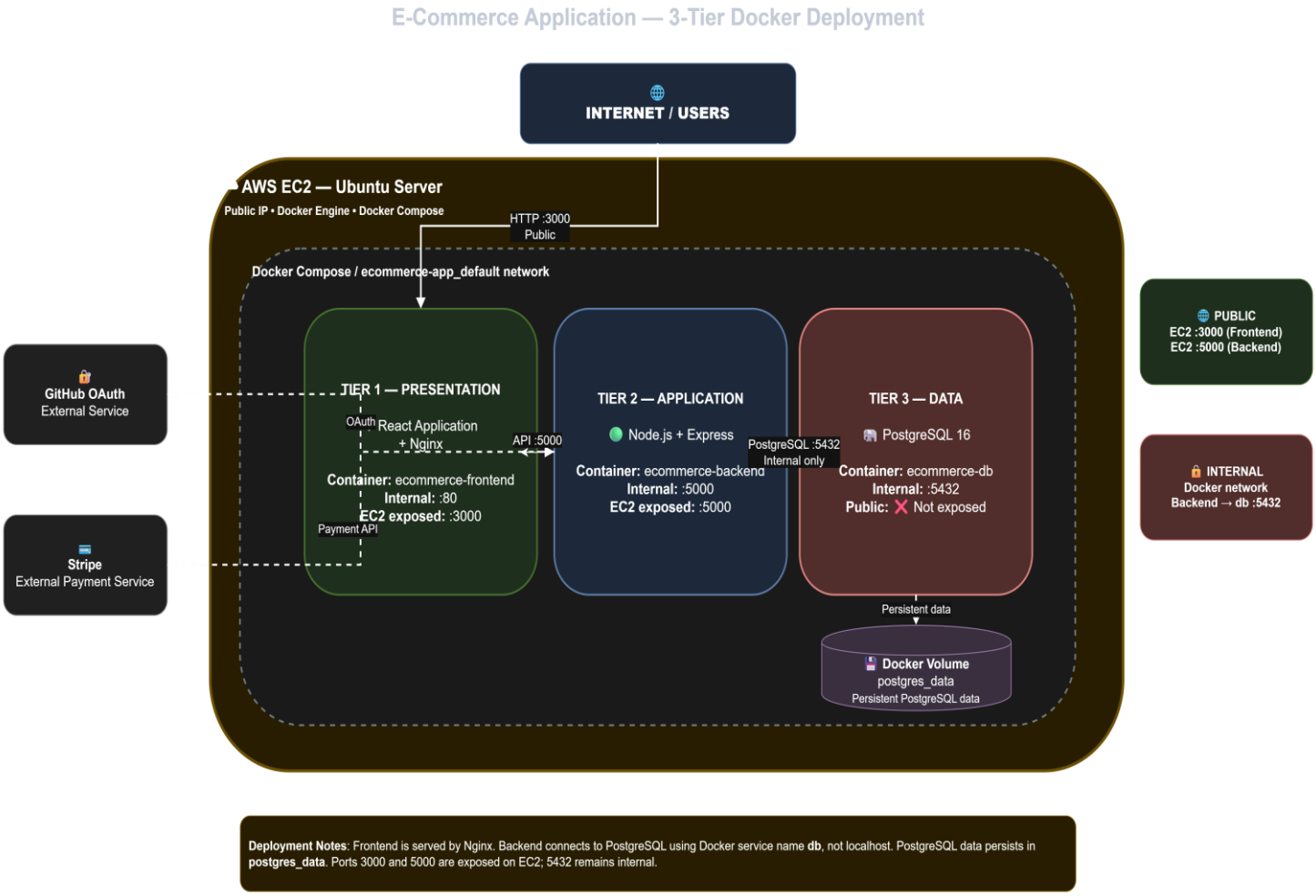
---

4. Existing Application Structure

Before containerization, the project consisted of a frontend and backend application.

```
ecommerce-app/
│
├── client/
│   ├── public/
│   ├── src/
│   ├── package.json
│   ├── Dockerfile
│   └── nginx.conf
│
├── server/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── index.js
│   ├── package.json
│   ├── Dockerfile
│   └── .dockerignore
│
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── package.json
└── README.md
```

5. System Architecture



External services:

The application also communicates with:

- GitHub OAuth for authentication
- Stripe for payment processing

---

6. Architecture Explanation

Tier 1 – Presentation Layer

- The frontend is developed using React.js.
- The React application is built into static production files and served using **Nginx**.
- The frontend container: ecommerce-frontend
- uses: React.js + Nginx
- Inside the Docker network, Nginx serves the application on port: 80
- The EC2 host exposes it on: 3000
- Therefore: EC2:3000 → Frontend Container:80

---

7. Tier 2 – Application Layer

- The backend is developed using: Node.js + Express.js

- The backend runs inside: ecommerce-backend

- The application listens on: 5000

- The EC2 host exposes: 5000

- Therefore: EC2:5000 → Backend Container:5000

- The backend communicates with PostgreSQL using the Docker Compose service name: db and

PostgreSQL port: 5432

****Important:**

The backend does not use localhost to connect to PostgreSQL. It uses the Docker service name db.

Example: postgresql://ecommerce_user:<password>@db:5432/ecommerce_db

---

8. Tier 3 – Data Layer

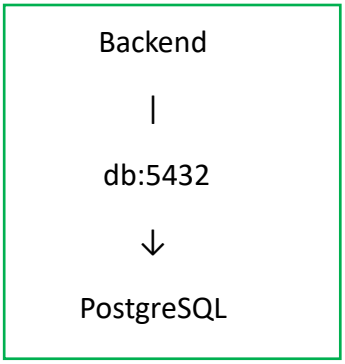
- The database uses: PostgreSQL 16

- The database container is: ecommerce-db

- The PostgreSQL container listens internally on: 5432

- For security, PostgreSQL is **not exposed to the public EC2 interface**.

- The backend accesses it internally through the Docker network.



---

9. Persistent Storage

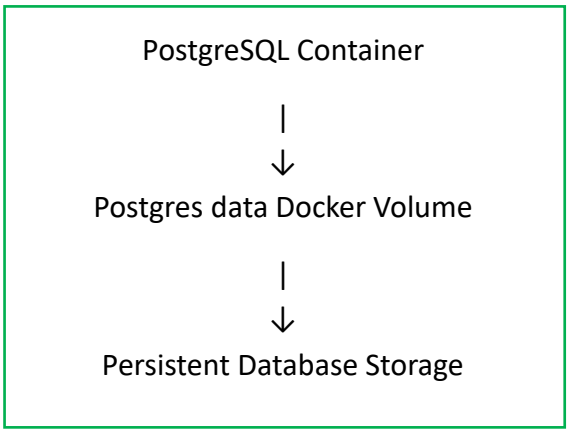
- A Docker named volume is used for PostgreSQL.

- Volume: postgres_data

- The volume is mounted to: /var/lib/postgresql/data

- This ensures that database data is not lost when the PostgreSQL container is recreated.

- Architecture:



---

10. Public vs Internal Ports

Service	Container port	EC2 port	Access
Frontend/Nginx	80	3000	Public
Backend/Node.js	5000	5000	Public
PostgreSQL	5432	Not Exposed	Internal

Public access

Frontend: http://<EC2-PUBLIC-IP>:3000

Backend: http://<EC2-PUBLIC-IP>:5000

Internal access

Backend → PostgreSQL:

db:5432

This prevents direct external access to the PostgreSQL database.

---

11. AWS EC2 Setup

The application was deployed on an AWS EC2 Ubuntu server.

Steps

1. Launch an EC2 instance.
2. Select Ubuntu as the operating system.
3. Connect to the server using SSH.
4. Install Docker.
5. Verify Docker installation.
6. Clone the application repository.
7. Configure environment variables.
8. Create Dockerfiles.
9. Create Docker Compose configuration.
10. Build and start the containers.

---

12. Clone the Repository

git clone https://github.com/tejswini-maingade/ecommerce-app.git

cd ecommerce-app

Verify the project: ls

Expected directories/files:

- Client
- Server
- package.json

---

13. Environment Configuration

- The backend uses a .env file for configuration.

Example structure:

```
PORT=5000
DB_URL=<database connection string>
SESSION_SECRET=<session secret>
FRONT_DOMAIN=<frontend URL>
SERVER_URL=<backend URL>
NODE_ENV=development

GITHUB_CLIENT=<GitHub client ID>
GITHUB_SECRET=<GitHub client secret>

STRIPE_PUBLIC=<Stripe public key>
STRIPE_SECRET=<Stripe secret key>
```

Security

Sensitive credentials are not committed to GitHub.

The .gitignore contains:

```
.env
node_modules/
.vercel
vercel.json
```

This prevents sensitive environment configuration from being accidentally pushed to the repository.

---

14. Dockerization

Separate Dockerfiles were created for the frontend and backend.

Backend Dockerfile

- The backend Docker image uses: node:22-alpine
- The application is copied into the container and dependencies are installed.
- The container starts the Node.js application.

---

15. Frontend Dockerization

The frontend uses a **multi-stage Docker build**.

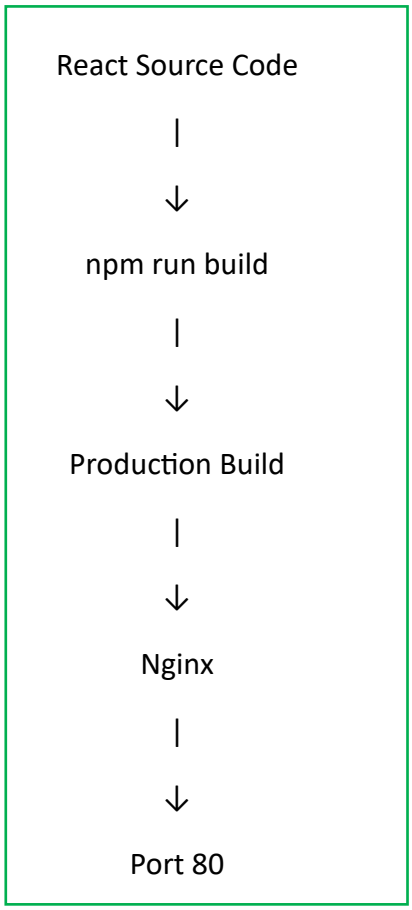
Build stage

React dependencies are installed and the production application is generated using: npm run build

Production stage

Nginx is used to serve the generated React production files.

Architecture:



This reduces the need to run the React development server in production.

---

16. Docker Compose

- Docker Compose is used to manage all application services.

- The services are:

backend

db

frontend

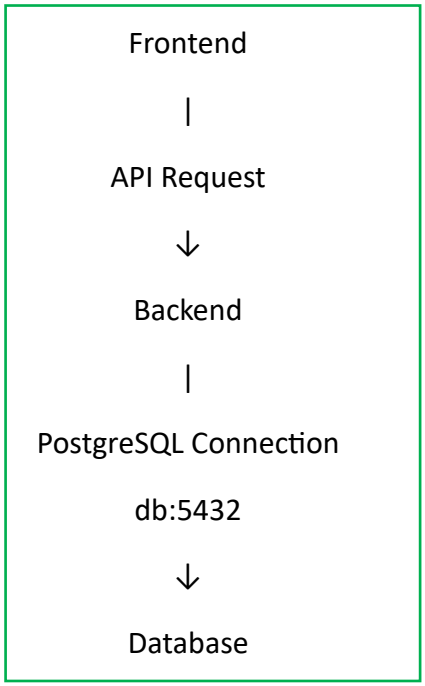
- Docker Compose creates a shared network: ecommerce-app_default

This allows containers to communicate using service names.

---

17. Docker Network Communication

- The containers communicate using the Docker Compose network.



The database does not need to be exposed publicly because the backend can reach it through the internal Docker network.

---

18. Build the Backend Image

The backend image was built using: **docker compose build backend**

or:

docker build -t ecommerce-backend:1.0 ./server

The Docker image was then used by Docker Compose.

---

19. Build the Frontend Image

- The frontend image was built using:

- **docker build -t ecommerce-frontend: 1.0 ./client**

- The frontend Dockerfile performs the React production build and serves the result using Nginx.

---

20. Start the Application

- The complete application can be started using:

- docker compose up -d --build

This command:

- Builds the required images
- Creates the Docker network
- Creates the PostgreSQL volume
- Starts the frontend
- Starts the backend
- Starts PostgreSQL

---

21. Verify Running Containers

Run: docker compose ps

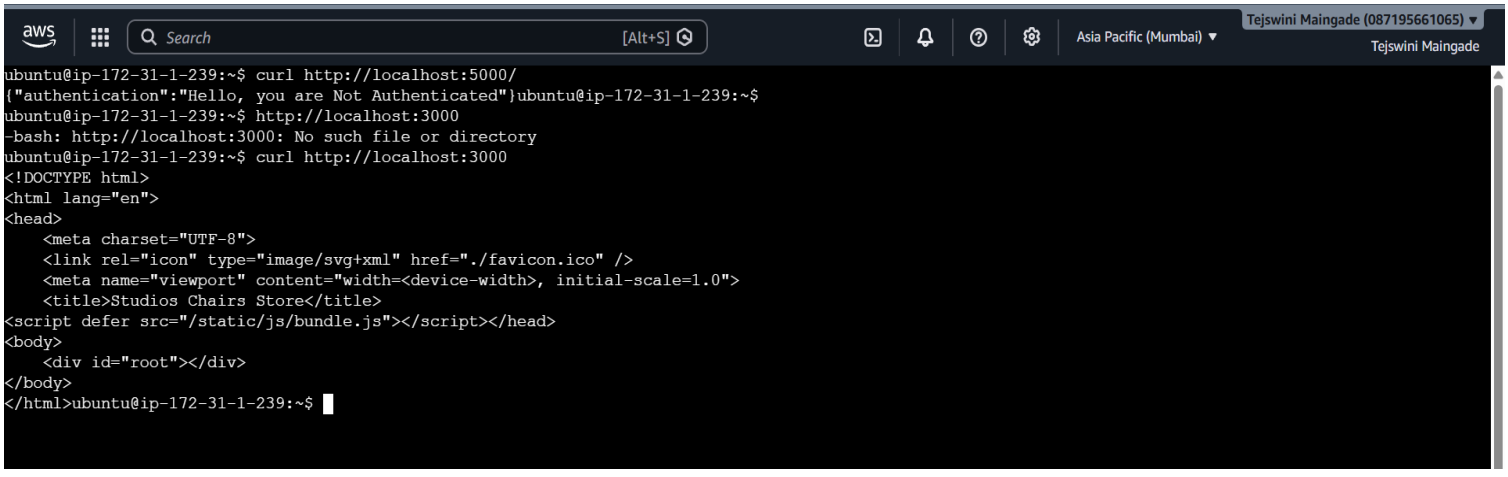
```
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker compose ps
NAME                IMAGE                   COMMAND                  SERVICE    CREATED              STATUS              PORTS
ecommerce-backend   ecommerce-app-backend   "docker-entrypoint.s…"   backend    About a minute ago   Up About a minute   0.0.0.0:5000->5000/tcp, [::]
000->5000/tcp
ecommerce-db        postgres:16-alpine      "docker-entrypoint.s…"   db         About a minute ago   Up About a minute   5432/tcp
ecommerce-frontend  ecommerce-app-frontend  "/docker-entrypoint.…"   frontend   About a minute ago   Up About a minute   0.0.0.0:3000->80/tcp, [::]:3
0->80/tcp
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker compose up -d
[+] Running 3/3
 ✓ Container ecommerce-db        Running                                                                                                  0.0
 ✓ Container ecommerce-backend   Running                                                                                                  0.0
 ✓ Container ecommerce-frontend  Running                                                                                                  0.0
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker exec ecommerce-db pg_isready -U ecommerce_user -d ecommerce_db
/var/run/postgresql:5432 - accepting connections
ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

---

22. Backend Health Check

The backend was verified using:

curl http://localhost:5000/



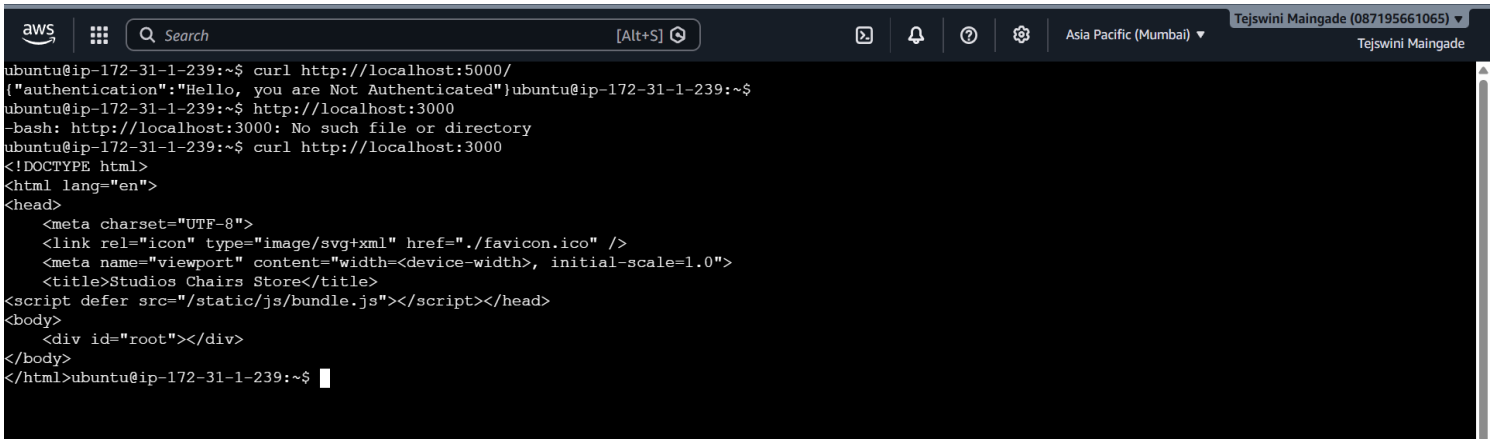
This confirms that the backend application is running and responding to requests.

---

23. Frontend Health Check

The frontend was verified using:

curl http://localhost:3000/



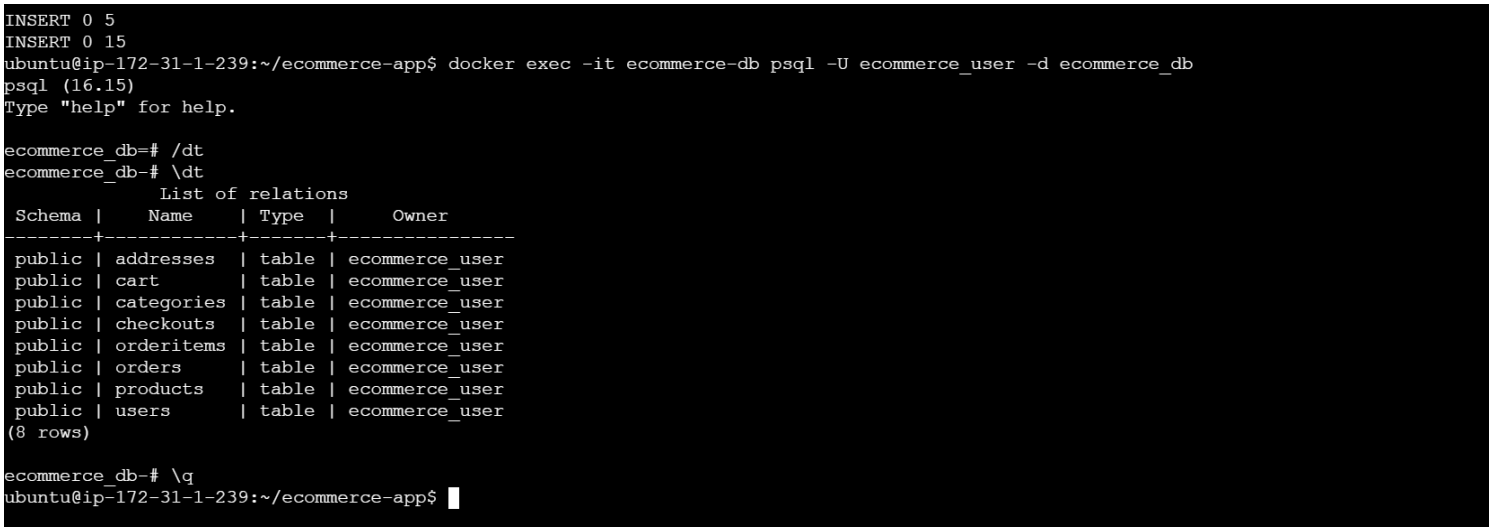
The command returned the React application's HTML document.

This confirms that Nginx is successfully serving the frontend production build.

---

24. Database Health Check

PostgreSQL was verified using: sudo docker exec ecommerce-db \pg_isready -U ecommerce_user -d ecommerce_db



This confirms that PostgreSQL is running and accepting connections.

---

25. Database Verification

The database contains the application tables.

Command: docker exec ecommerce-db \psql -U ecommerce_user -d ecommerce_db -c "\dt"

The deployment contains:

addresses
cart
categories
checkouts
order items
orders
products
users

This confirms that the application database schema has been successfully initialized.

---

26. Product Data Verification

- The application database was also verified for product data.

Command: docker exec ecommerce-db \psql -U ecommerce_user -d ecommerce_db \ -c "SELECT COUNT(*) FROM products;"

Result:

15

This confirms that product records are available in the database.

---

27. Application Access

The deployed application can be accessed through the EC2 public IP.

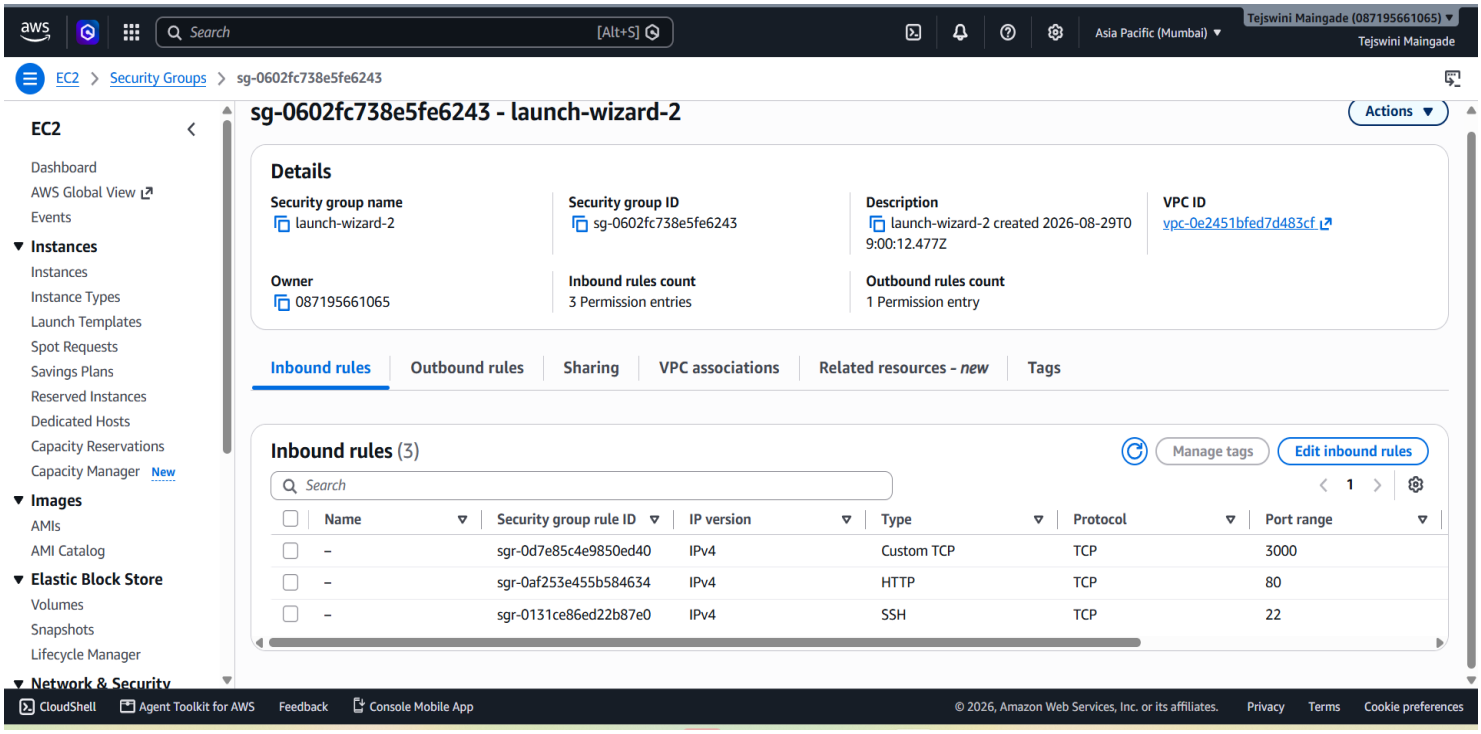
Frontend

http://<EC2-PUBLIC-IP>:3000

Backend

http://<EC2-PUBLIC-IP>:5000

The AWS Security Group should allow the required application ports.



---

28. Troubleshooting Performed

- This section is **very valuable for a DevOps practical exam** because it demonstrates real troubleshooting rather than only successful commands.

Issue 1 – Docker Permission Denied

Initially:

- permission denied while trying to connect to the Docker API
- The issue was related to Docker socket permissions.
- Docker commands were executed with appropriate privileges while troubleshooting.

Issue 2 – PostgreSQL Port Conflict

- Initially PostgreSQL attempted to bind: 0.0.0.0:5432

but the port was already in use.

Error:

- failed to bind host port 0.0.0.0:5432/tcp: address already in use

Resolution

- PostgreSQL was configured to remain available through the Docker network without exposing host port 5432.
- This is also a better architecture because the database does not require public access.

---

29. Issue 3 – Docker Build Context

- An incorrect Docker build command initially resulted in:

COPY failed: no source files were specified

- The issue was caused by the Dockerfile referencing paths incorrectly relative to the build context.
- The Dockerfile and build context were corrected so that:

```
server/
 ├── Dockerfile
 ├── package.json
 └── application files
```

could be copied correctly.

---

30. Issue 4 – Frontend Build Disk Space

- During the frontend Docker build, the following error occurred: ENOSPC: no space left on device

- The EC2 root filesystem was completely full:

```
/dev/root        6.7G                    100%
```

- Docker storage was investigated using: docker system df
- The filesystem usage was investigated using: sudo du -xhd1 / 2>/dev/null | sort -h
- The EC2 storage was then extended to provide sufficient disk space for the Docker build.
- After increasing available storage, the frontend build completed successfully.

---

31. Final Deployment Status

After resolving the deployment issues, all three containers were successfully running.

ecommerce-frontend

ecommerce-backend

ecommerce-db

The final environment was verified using:

docker compose ps

```
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker compose ps
NAME                IMAGE                    COMMAND                  SERVICE    CREATED              STATUS              PORTS
ecommerce-backend   ecommerce-app-backend    "docker-entrypoint.s…"   backend    About a minute ago   Up About a minute   0.0.0.0:5000->5000/tcp, [::]
000->5000/tcp
ecommerce-db        postgres:16-alpine       "docker-entrypoint.s…"   db         About a minute ago   Up About a minute   5432/tcp
ecommerce-frontend  ecommerce-app-frontend   "/docker-entrypoint.…"   frontend   About a minute ago   Up About a minute   0.0.0.0:3000->80/tcp, [::]:3
0->80/tcp
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker compose up -d
[+] Running 3/3
 ✓ Container ecommerce-db        Running                                                                                    0.0
 ✓ Container ecommerce-backend   Running                                                                                    0.0
 ✓ Container ecommerce-frontend  Running                                                                                    0.0
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker exec ecommerce-db pg_isready -U ecommerce_user -d ecommerce_db
/var/run/postgresql:5432 - accepting connections
ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

The frontend responded on:

3000

The backend responded on:

5000

PostgreSQL was available internally on:

5432

---

32. Security Considerations

The following security practices were implemented:

Environment variables

Sensitive credentials are stored in: server/.env

and excluded from Git using: .env

Database

PostgreSQL port 5432 is not exposed publicly.

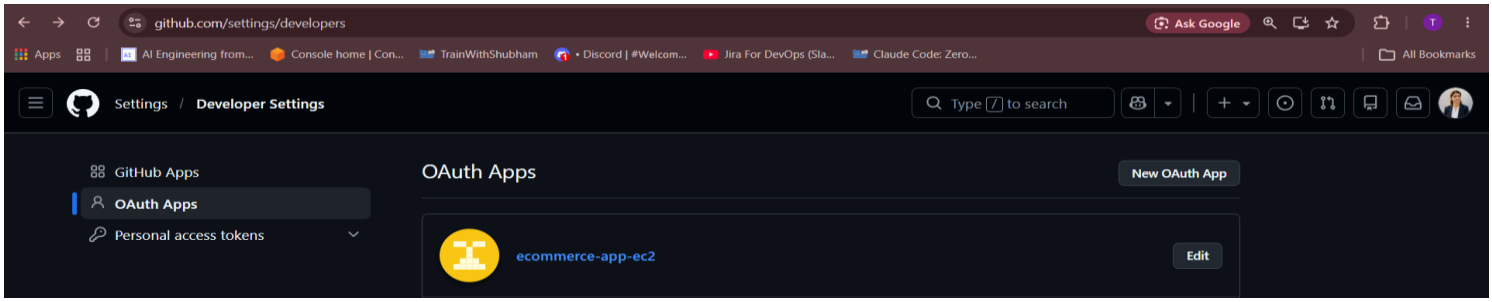
Only the backend accesses PostgreSQL through the Docker network.

Docker Network

Internal service communication uses Docker's private network.

Secrets

GitHub OAuth and Stripe credentials are configured through environment variables instead of hardcoding them in the application.



---

33. DevOps Concepts Demonstrated

This project demonstrates practical knowledge of:

- AWS EC2
- Linux administration
- Git/GitHub
- Docker
- Dockerfiles
- Multi-stage Docker builds
- Docker Compose
- Docker networking
- Docker volumes
- Containerized PostgreSQL
- Nginx
- Environment variables
- Application health checks
- Database verification
- Troubleshooting
- Cloud deployment

---

34. Challenges and Learnings

Challenges

- Docker permission issues
- PostgreSQL host port conflict
- Incorrect Docker builds context
- Frontend Docker build taking significant time
- EC2 disk space exhaustion
- Database connectivity between containers

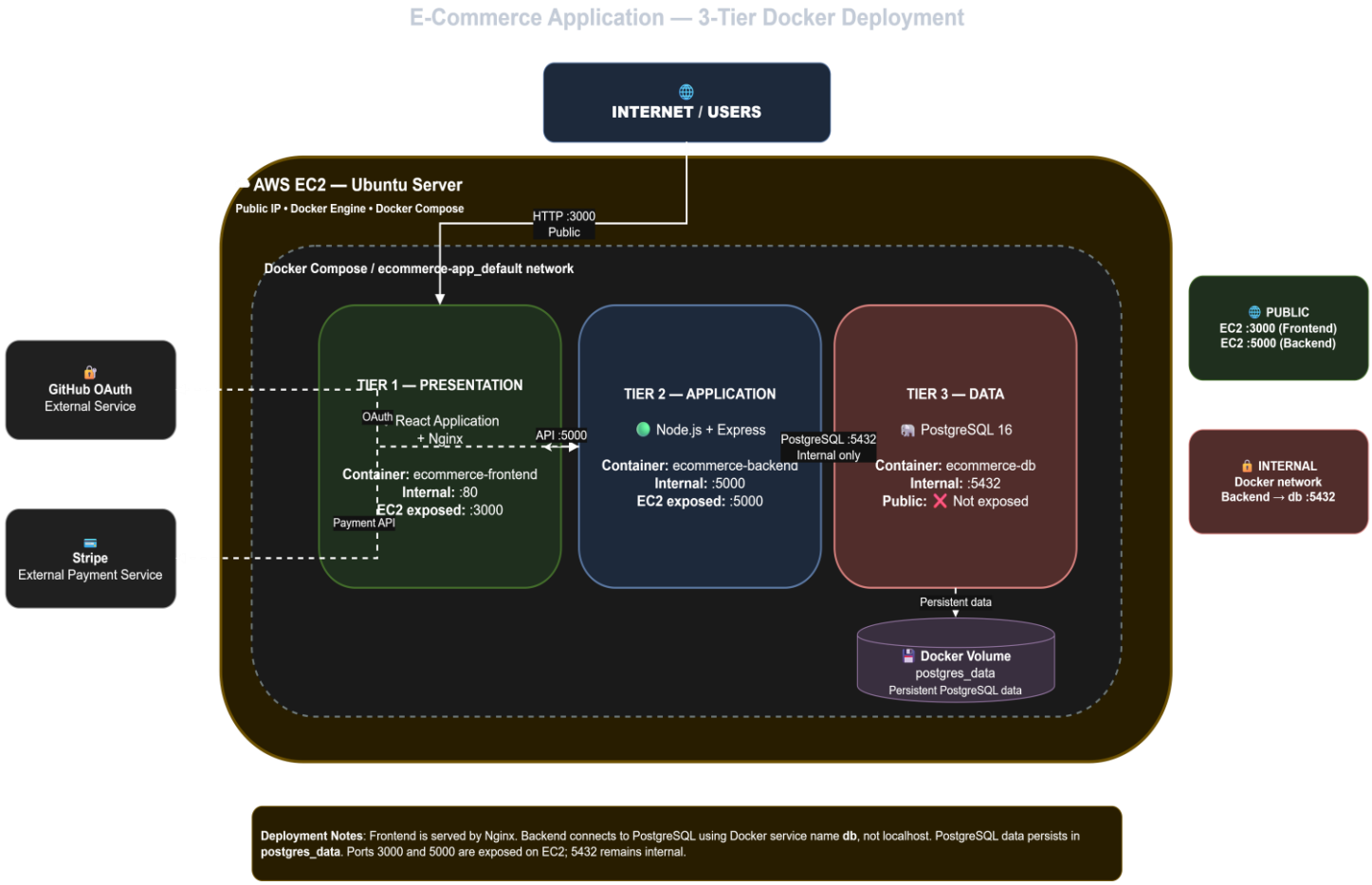
Key learnings

I learned that successful container deployment requires more than simply creating Dockerfiles. Networking, storage, environment variables, port exposure, security, and troubleshooting are equally important.

The project provided hands-on experience in taking an existing full-stack application and converting it into a **cloud-deployed multi-container architecture**.

---

35. Final Architecture Flow

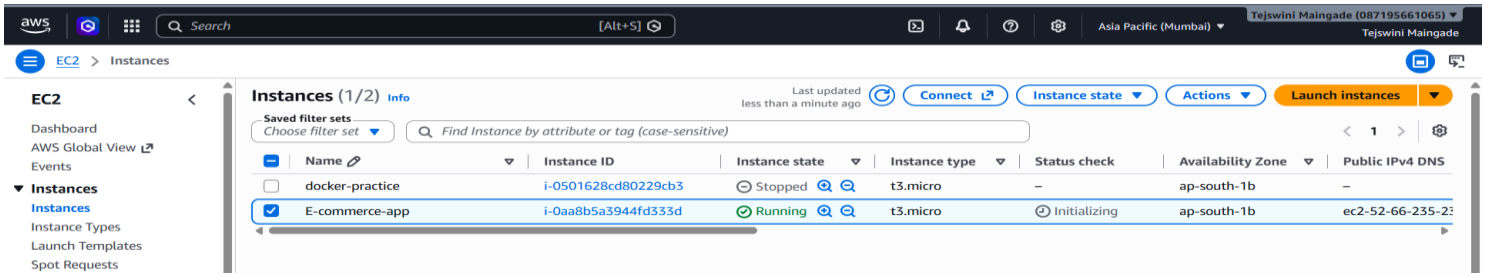


---

36. Screenshots to Include

Screenshot 1

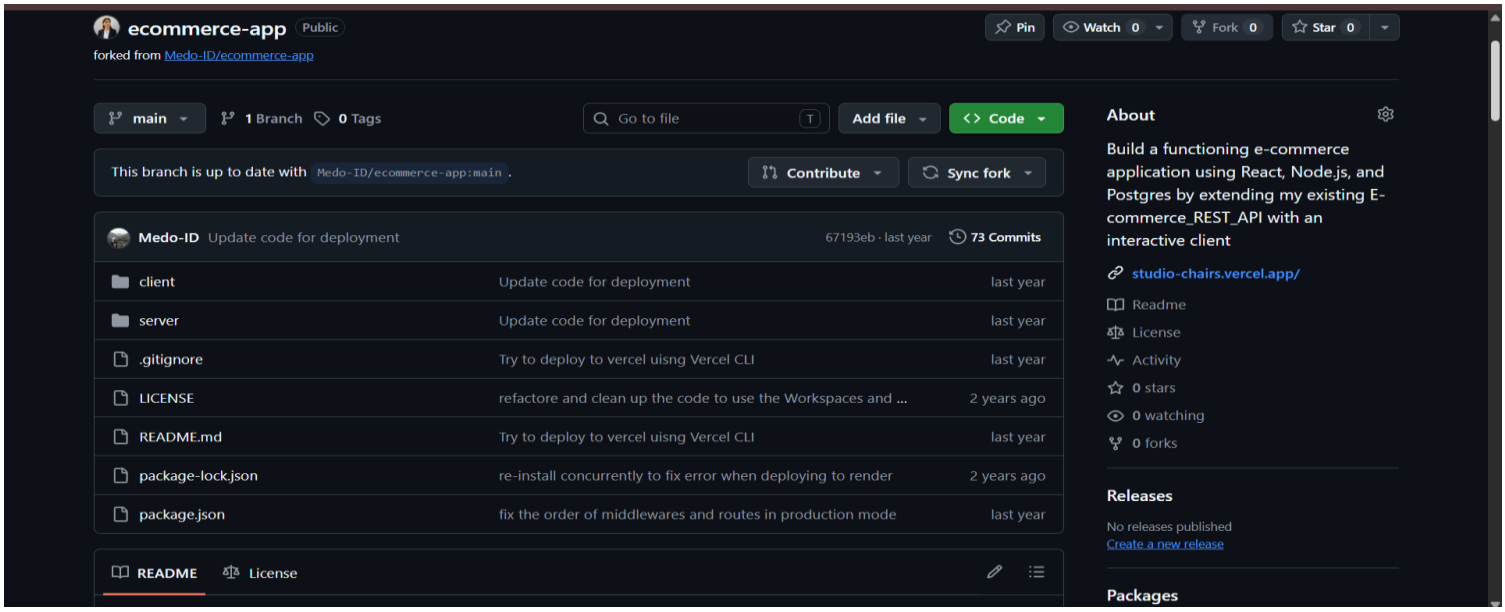
AWS EC2 instance



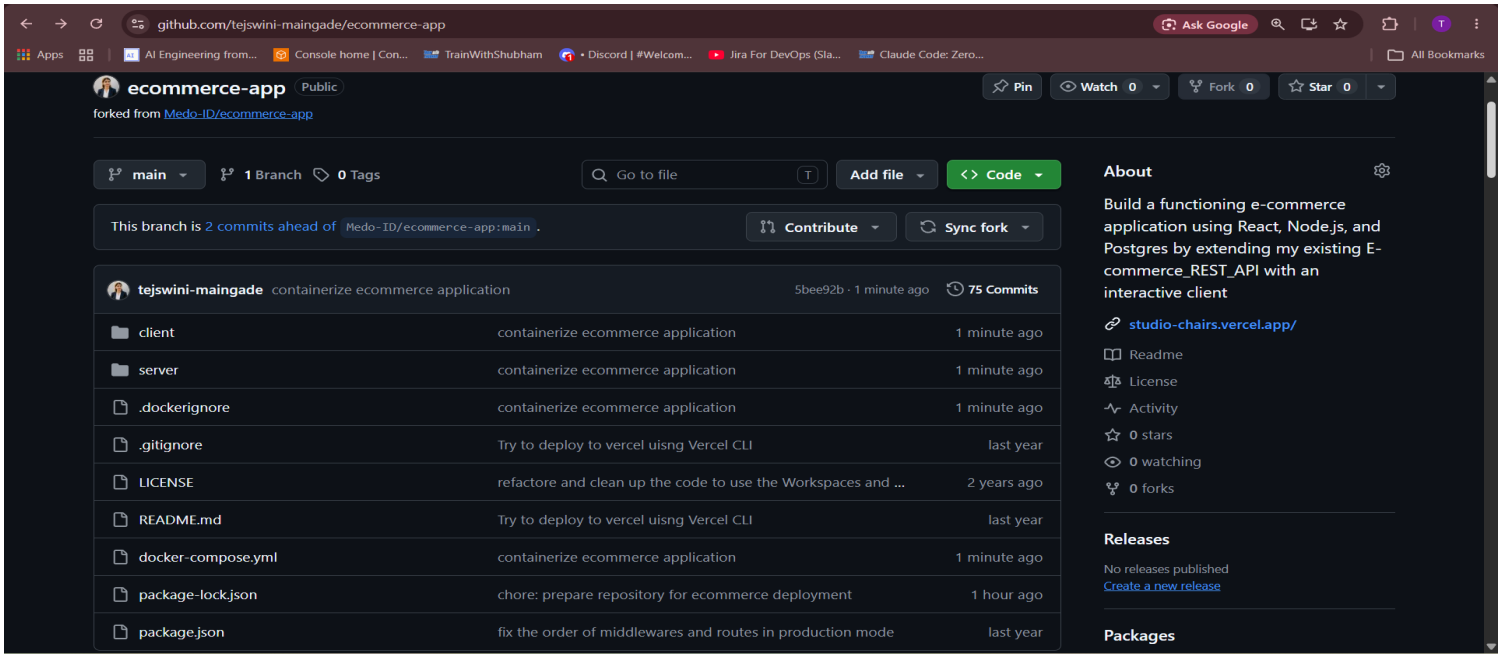
Screenshot 2

Project Repository

Before Docker



After Docker



Screenshot 3

Docker version

docker --version

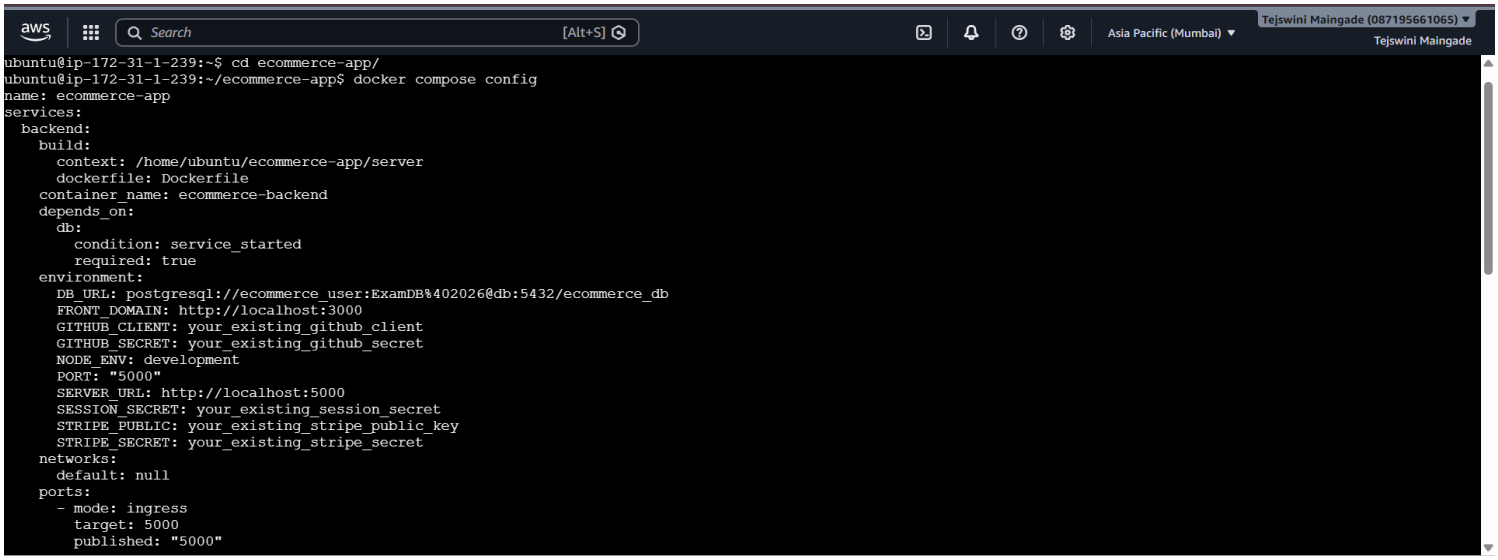
docker compose version

```
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

Screenshot 4

Docker Compose configuration

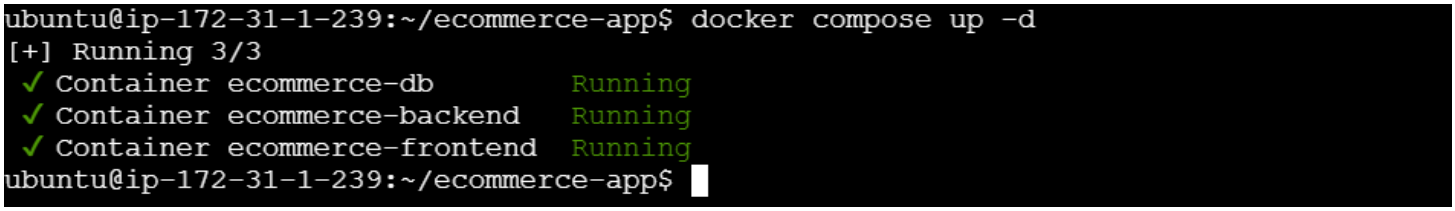
docker compose config



Screenshot 5

Successful build

docker compose up -d



Screenshot 6

Running containers

docker compose ps

Screenshot 7

Backend health check

curl http://localhost:5000/

Screenshot 8

Frontend health check

curl http://localhost:3000/

Screenshot 9

Database health

sudo docker exec ecommerce-db \pg_isready -U ecommerce_user -d ecommerce_db

```
ubuntu@ip-172-31-1-239:~/ecommerce-app$ sudo docker exec ecommerce-db \pg_isready -U ecommerce_user -d ecommerce_db
/var/run/postgresql:5432 - accepting connections
ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

Screenshot 10

Database tables

docker exec ecommerce-db \psql -U ecommerce_user -d ecommerce_db -c "\dt"

```
INSERT 0 5
INSERT 0 15
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker exec -it ecommerce-db psql -U ecommerce_user -d ecommerce_db
psql (16.15)
Type "help" for help.

ecommerce_db=# /dt
ecommerce_db-# \dt
              List of relations
 Schema |    Name    | Type  |     Owner
--------+------------+-------+----------------
 public | addresses  | table | ecommerce_user
 public | cart       | table | ecommerce_user
 public | categories | table | ecommerce_user
 public | checkouts  | table | ecommerce_user
 public | orderitems | table | ecommerce_user
 public | orders     | table | ecommerce_user
 public | products   | table | ecommerce_user
 public | users      | table | ecommerce_user
(8 rows)

ecommerce_db-# \q
ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

Screenshot 11

Product count

docker exec ecommerce-db psql -U ecommerce_user -d ecommerce_db -c "SELECT COUNT(*) FROM products;"

```
ubuntu@ip-172-31-1-239:~/ecommerce-app$ docker exec ecommerce-db psql -U ecommerce_user -d ecommerce_db -c "SELECT COUNT(*) FROM products;"
 count
-------
    15
(1 row)

ubuntu@ip-172-31-1-239:~/ecommerce-app$
```

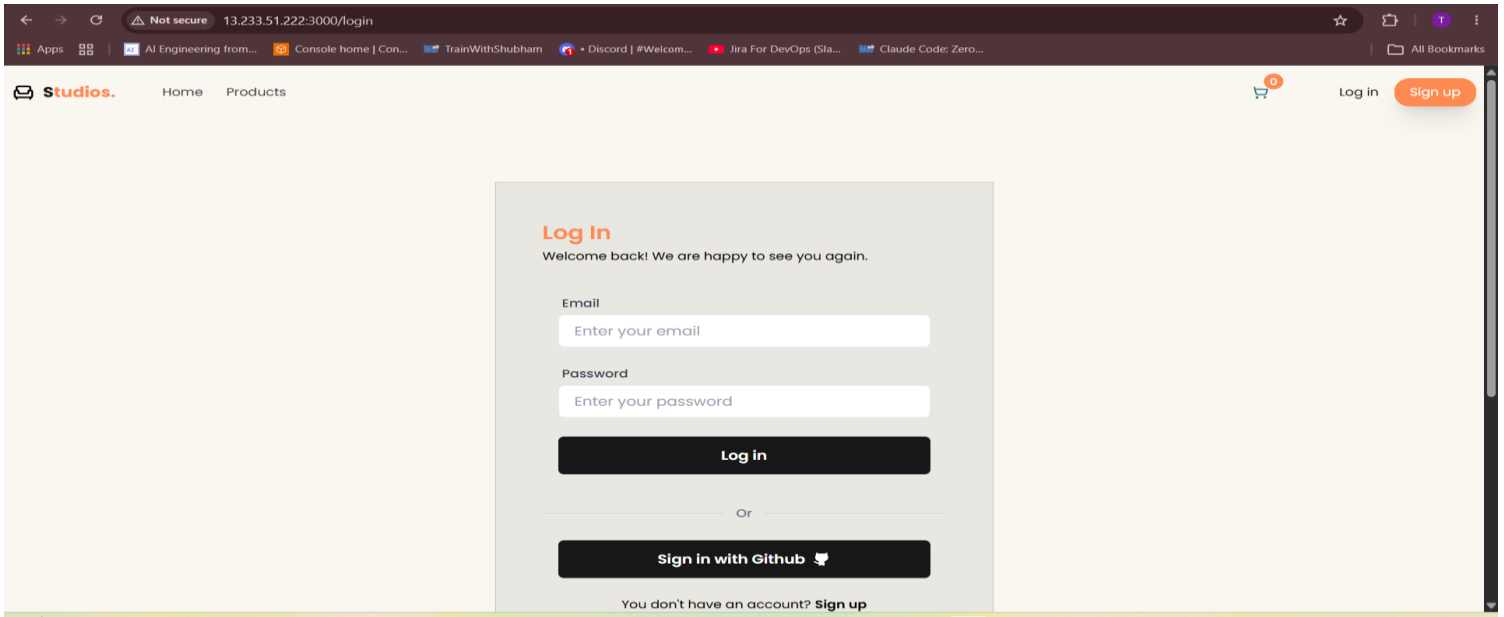
Screenshot 12

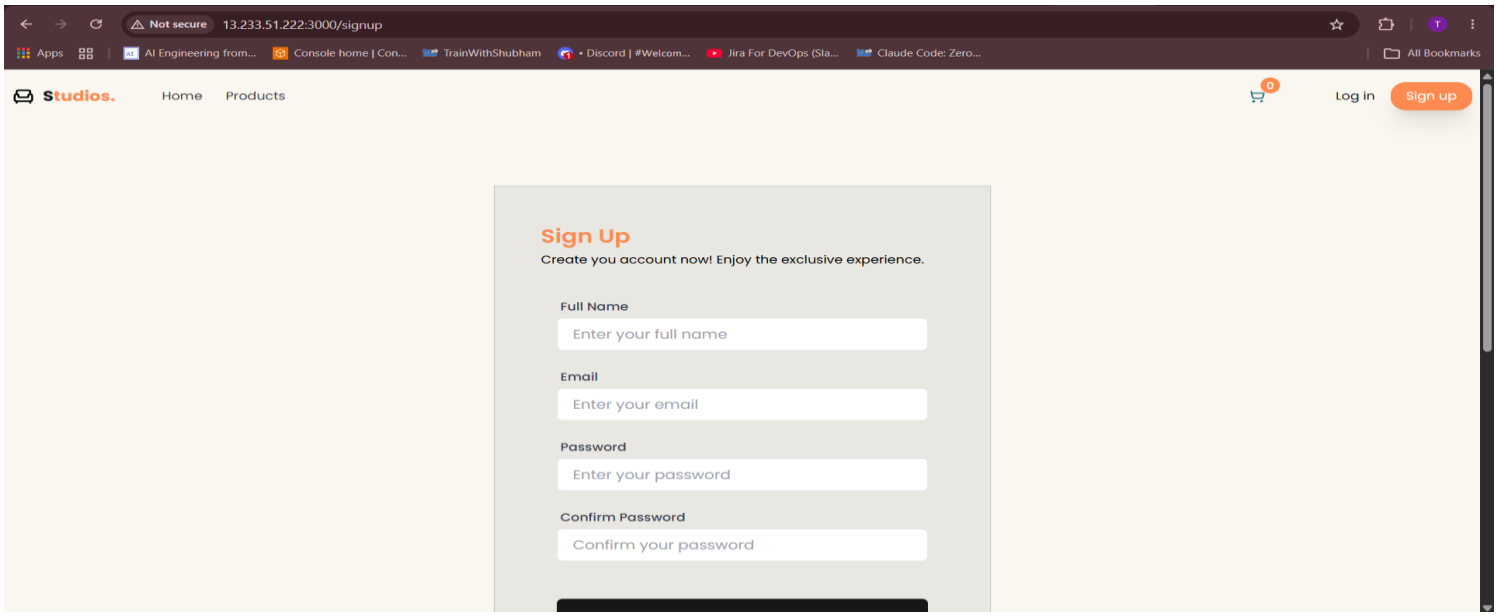
Browser showing the actual E-Commerce application

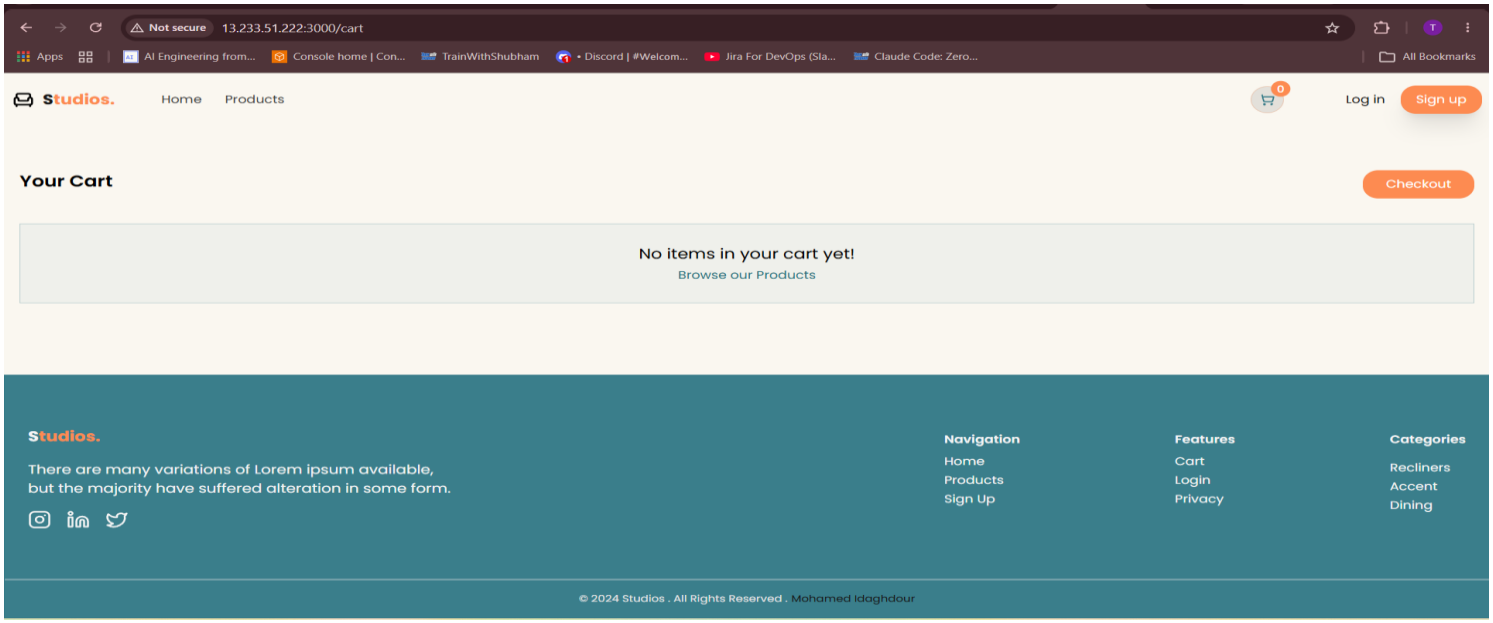
Show:

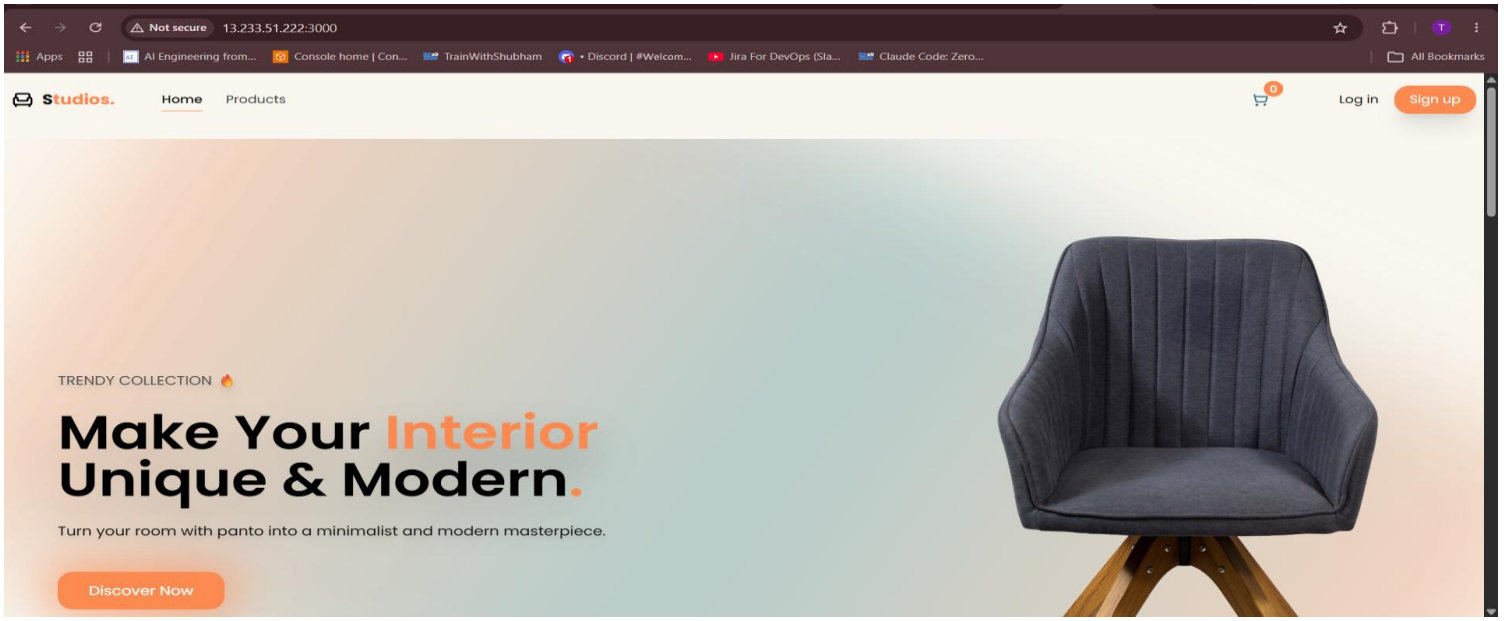
http://<EC2-PUBLIC-IP>:3000

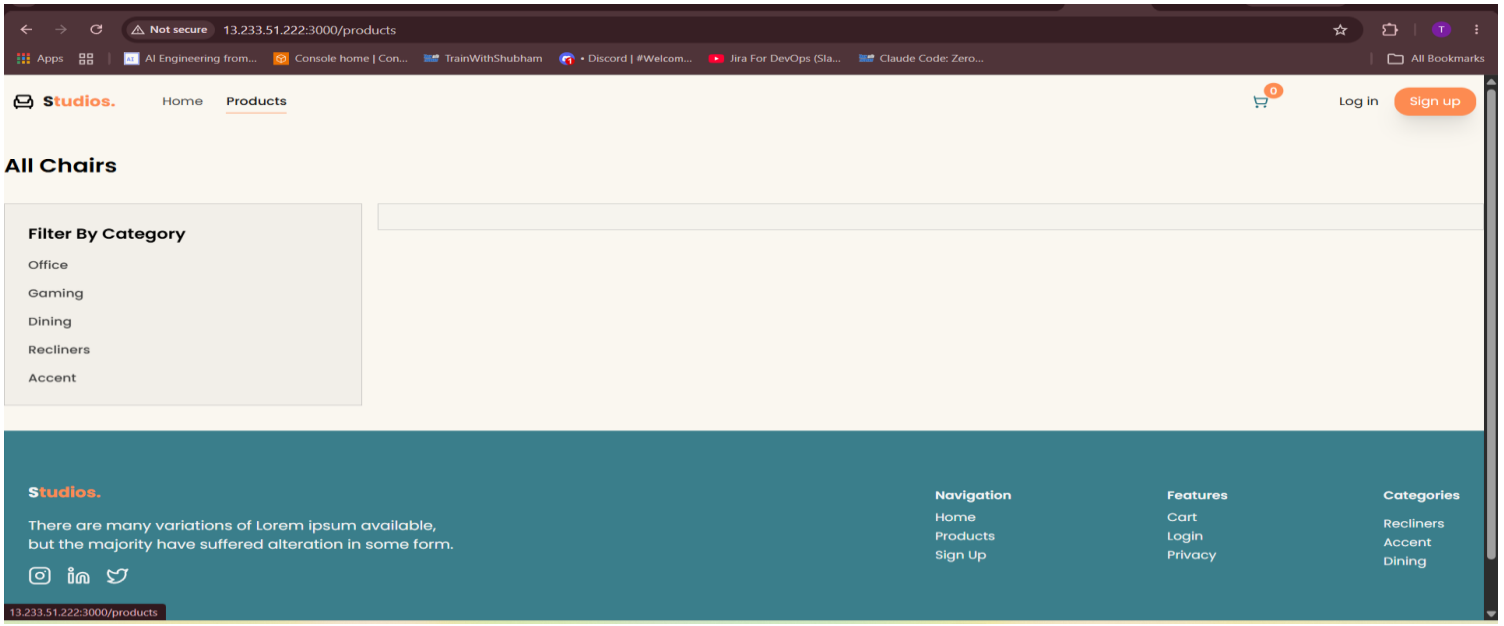
with the application loaded.


Not secure 13.233.51.222:3000/login
Studios.
Home
Products
Log in
Sign up
Log In
Welcome back! We are happy to see you again.
Email
Enter your email
Password
Enter your password
Log in
Or
Sign in with Github
You don't have an account? Sign up


Not secure 13.233.51.222:3000/signup
Studios.
Home
Products
Log in
Sign up
Sign Up
Create you account now! Enjoy the exclusive experience.
Full Name
Enter your full name
Email
Enter your email
Password
Enter your password
Confirm Password
Confirm your password

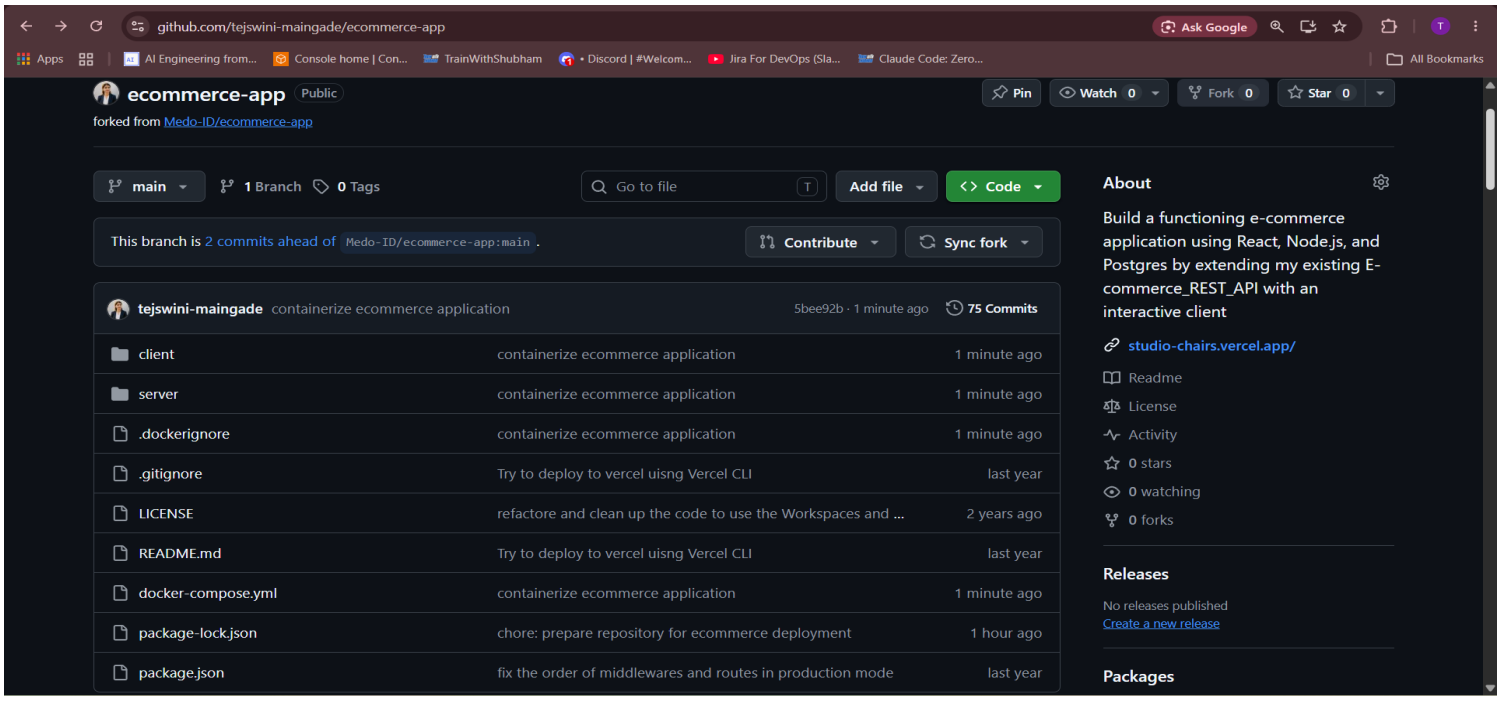

Not secure 13.233.51.222:3000/cart
Studios.
Home
Products
Log in
Sign up
Your Cart
Checkout
No items in your cart yet!
Browse our Products
Studios.
There are many variations of Lorem ipsum available, but the majority have suffered alteration in some form.
Navigation
Home
Products
Sign Up
Features
Cart
Login
Privacy
Categories
Recliners
Accent
Dining
© 2024 Studios . All Rights Reserved . Mohamed Idaghdour





---

37. GitHub Repository



---

38. Conclusion

The E-Commerce application was successfully containerized and deployed on an AWS EC2 Ubuntu server using Docker and Docker Compose.

The final architecture consists of three separate containers for the frontend, backend, and PostgreSQL database. Docker networking enables communication between the application tiers, while a persistent Docker volume protects PostgreSQL data.

The deployment was validated through application, database, and container-level health checks.

The project provided practical experience with **containerization, cloud deployment, networking, persistent storage, security configuration, and real-world troubleshooting**, which are essential DevOps skills.

---

39. Thank You

Special Thanks

A special thanks to **Shubham Londhe** for his continuous guidance, support, and for creating a learning environment that encourages us to build, practice, troubleshoot, and grow as DevOps professionals.

🙏 **Thank you for the guidance and motivation throughout this learning journey.**
